Default Arguments

PHP allows you to assign default values to function parameters when the function is declared. These are known as "default arguments." Using default values means that the calling code doesn't need to pass a value and will not generate an error if the parameter is missing. If calling code does pass a value, then the default is ignored and the passed-in value is used. Let's look at a simple example.

```
function addOneOrMore($a, $b = 1) {
    return $a + $b;
}

var_dump(addOneOrMore(2, 2));
var_dump(addOneOrMore(2));

Output:
int(4)
int(3)
```

We define a function `addOneOrMore()` with takes two parameters `$a` and `$b` and returns the sum of them. The second argument `$b` has a default value of 1. If `addOneOrMore()` is called without a second value, it uses the default, as we can see from the output. It is generally good practice to place arguments with default values at the end of the parameter list. Remember the "weird quirk" I mentioned, concerning nullable types? If a function parameter is declared with a default value of `null` then it is made implicitly nullable. `function foo (string $foo = null) ...` is implicitly `function foo(?string $foo = null) ...` and in version 8.4 this will emit a deprecation notice. It is scheduled for removal in version 9 and any parameter with a `null` default value must be explicitly made nullable.

Variadic Arguments

A function that supports variable numbers of arguments is called a *variadic* function. Some built-in PHP functions support variable numbers of arguments, a good example is `sprintf()`, which takes a formatting string as its first argument, and an arbitrary amount of strings after that with the caveat that the number of strings supplied must match the specified format: